PYTHON

Basic

PYTHON

It is the language that will be used in the course, although there are many more. It is easy to use and very profitable for the programs we will create. It also has many libraries and there is high demand in the job market.

TIPS:

Beginners:

- Do not try to make everything perfect the first time.
- You can ask Google or ChatGPT about what you do not understand.
- On Slack you can ask:

Present the code that does not work and ask how to fix it.

- Visit the official Python website.



IMPORTANT IN

a program:

```
# Calculate the absolute value
x = -1
if x < 0:
    y = -x
else:
    y = x

print(y)
```

```
1
```

- # is used to make comments.
- : is used before starting a block of code for a loop or condition. After the colon, what follows is a process (block of code).
- It must be indented with 4 spaces before the starting word of the process.

These last two are like the spelling of code, called Syntax.

FUNCTIONS AND

what they do

Functions are processes that Python already has defined with a keyword. Here are the most used:

- **print():** Displays a message or data on the screen.
- **len():** Counts the number of elements in a list, string, or collection.
- **input():** Allows the user to enter data from the keyboard.
- **type():** Tells you what type of data an object is (text, integer, decimal, etc.).
- **int(), str(), float():** Convert one data type into another (type conversion).

MATHEMATICAL

```
# Addition
print(2 + 3)
```

```
5
```

operators

+ Addition
- Subtraction
***** Multiplication
/ Division
// Integer division
****** Exponentiation
% Remainder of division

In Python there is precedence (order of operations):

1. Parentheses
2. Exponentiation
3. Multiplication
4. Division
5. Addition
6. Subtraction

COMPARISON

operators

```
# Addition
print(2 + 3)
```

```
5
```

Used in Boolean values or conditionals.

> Greater than
< Less than
>= Greater or equal
<= Less or equal
== Equality
!= Not equal

VARIABLES:

What are they?

sThey are like boxes or containers that store data.
Important:
= Assignment
== Equality

Assignment:

Giving a value or data to a variable.

Example:
x = 7
y = ‘Python’

Overwrite:

A variable can change its value if it is overwritten throughout the code.
Example:
x = 2
x = x + 2
x would be 4

Important:

It is recommended not to use complicated names. Better to use letters, avoid names of Python functions, and the name cannot start with a number.

DATA TYPES:

int = Integer

Integer values used in addition, subtraction, multiplication.
Example:
}x = 2
print(type(x)) **<class 'int'>**

float = Decimal

Float are values of decimal numbers, can be seen in any operation but division is always a float.
Example:
x = 1.0
print(type(x)) **<class ‘int?>**

list = List

Data type used so that many values are in a determined order. Uses brackets and separated by commas.
Example:

x = [50, 60, 70]
print(type(x)) **<class ‘list’>**

str = Text

Str or String are for text values, can be seen in a variable that seeks a name; must always go with (‘’) quotes either single or double (“”).
Example:
x = ‘Python’
print(type(x)) **<class ‘str’>**

bool = Boleano

Bool or Boolean value is when true or false is given, it is a conditional similar to yes or no.
Example:
yes = True
print (type(yes)) **<class ‘bool’>**
no = False **<class ‘bool’>**
print (type(no))

INDICES:

1. Indices are like the numbering of each value of the element.
Element = Value in the variable.

2. Enumeration starts at 0.
Example:
If it is a word like Apple it would be: A = 0, P = 1, P = 2, L = 3, E = 4.

3. It is used to access a specific data.

FLOAT:

```
num = 1 + 1.0
print(type(num))
```

```
<class 'float'>
```

If added with an integer (int) it results in a decimal (float).

```
num = 1 / 1
print(type(num))
print(num)
```

```
<class 'float'>
1.0
```

In a division it always gives a decimal value (float).

STR:

```
str1 = 'Hello'
str2 = 'World'
print(str1 + "," + str2)  # Adding a comma between
```

```
Hello,World
```

Join words, it is better to use the comma.

```
# Length of the string
string = 'Python'
print(len(string))
```

```
6
```

See the number of letters the text has.

```
# Repeat
text = "Hello "
repeated = text * 3
print(repeated)
```

```
Hello Hello Hello
```

Repeat text.

```
# Get the first character
text = "Apple"
part = text[0]  # 0th index
print(part)
```

Search only a part of the text.

TYPE CONVERSION:

Change the data type of the variable with: int(), float(), str(), list()

Example: It's seen that in the first a str data is inserted since it has quotes, then the same element is converted into float and in the last it is converted into int (integer, that is why decimal numbers are not seen).

```
# str type
x = '3.14'  # Not a numerical value but a string
print(x)
print(type(x))
```

```
3.14
<class 'str'>
```

```
# Converting from str type to float type
y = float(x)
print(y)
print(type(y))
```

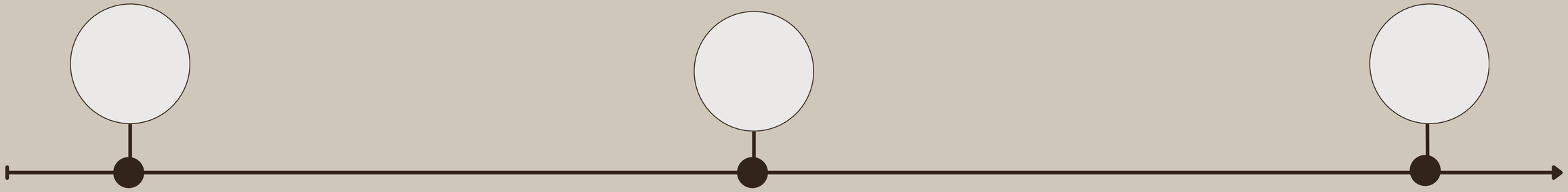
```
3.14
<class 'float'>
```

```
# Convert from float to int
z = int(y)
print(z)
print(type(z))
```

```
3
<class 'int'>
```

COLLECTIONS:

Allow storing different values in a variable.



LISTS:

Manage values in an ordered way. Any type of data can be used but without varying in the same list. To add you can use .append or to remove .remove, written with brackets.

BOTH:

Separated with commas, you can extract specific data with the index. Extracting is known as slicing. You can also do numerical operations with functions like: len(), sum(), max(), min().

TUPLES:

Store formatted elements of different data types. You can have several types of data in the same tuple, cannot add or remove elements, written with parentheses.

LISTS:

```
# Assigning a 2D list
table = [['3/24', '4/24', '5/24'], [80, 100, 90]]
```

		(Column 0)	(Column 1)	(Column 2)
(Row 0)	Date	3/24	4/24	5/24
(Row 1)	Score	80	100	90

```
# Extracting element at row 0, column 2
print(table[0][2])
```

```
24/5
```

```
# Extracting element at row 1 column 1
print(table[1][1])
```

```
100
```

2D List: It is like a table, extracts data by column and row index.

```
# Extracting the second element of the list
print(menu[1])
```

```
Soup
```

```
# Extracting the last element of the list
print(menu[-1])
```

```
# Change the last element of the list
menu[-1] = 'pudding'
print(menu)
```

```
['Salad', 'Soup', 'Steak', 'pudding']
```

We observe that elements are extracted by their index and also the last element is changed.

TUPLES:

```
# Example 1 using parentheses
x = (1, 2.0, '3')
print(x)
```

```
# Example 2: Omitting parentheses
x = 1, 2.0, '3'
print(x)
```

You can omit the parentheses and it is still the same process.

```
# Get the reference of elements
x = (1, 2.0, '3')
print(x[0])
```

```
1
```

```
# Slice (reference)
x = (1, 2.0, '3')
print(x[0:2])
```

```
(1, 2.0)
```

Search specific data.
Example: (x[0:2]) means data is searched from index 0 (beginning) up to before reaching index 2, that is why '3' does not appear.

BIBLIOTECAS

Collection that associates a key label to each element of a list, known as associative array in other languages. Data is stored as:
<key: element >
Thus we can access elements through the key words. Same as lists we can add and extract specific data.

```
# Dictionary
menu = {'apple':100, 'banana':80, 'orange':120}
print(menu)

# Accessing elements
print(menu['apple'])
```

If a value is assigned to a key word that is not in the dictionary it is added as a new element. To **add** a new element use add and to delete an element use **del**.

```
# Add a key and value
menu['grape'] = 150
print(menu)

# Remove a key and value
del menu['apple']
print(menu)
```

You can retrieve key, values and items with:

```
# Get keys
x = {'a': 1, 'b': 2, 'c': 3}
print(x.keys())

dict_keys(['a', 'b', 'c'])

# Get elements
x = {'a': 1, 'b': 2, 'c': 3}
print(x.values())

dict_values([1, 2, 3])

x = {'a': 1, 'b': 2, 'c': 3}
print(x.items())

dict_items([('a', 1), ('b', 2), ('c', 3)])
```

I HOPE IT

helps you.

REFERENCES:

- Google Colab. (2026). Python explanation. Retrieved from https://drive.google.com/drive/u/0/folders/1aEvVLsSTs48MRR1FTBu7BCXGI2XG2uI5
- Google Colab. (2026). Python exercises. Retrieved from https://drive.google.com/drive/u/0/folders/1aEvVLsSTs48MRR1FTBu7BCXGI2XG2uI5
- PDF Document. (2026). Python explanation. Retrieved from https://drive.google.com/drive/u/0/folders/1aEvVLsSTs48MRR1FTBu7BCXGI2XG2uI5
- YouTube. (2026). Class playlist: Basic Python – GCI World. Retrieved from https://www.youtube.com/playlist?list=PLiEF5y87wtyyc3mVmmRpUcAi_a74w52CI
- Zoom. (2026). Online class on Basic Python – GCI World. Retrieved from https://us06web.zoom.us/j/89296868328?pwd=tE16f24MbryVOTIrEnoXNPxEKYM
- Michelle. (2026). Basic Python 1. Canva. https://www.canva.com
- Microsoft Copilot. (2026, September 2). Guidance on citations and references [Artificial intelligence tool]. Microsoft.
- Google Translate. (2026). Text translation. Retrieved from https://translate.google.com
- According to Michelle (2026), in her presentation Uso adecuado de la IA (translated by Microsoft Copilot, 2026)